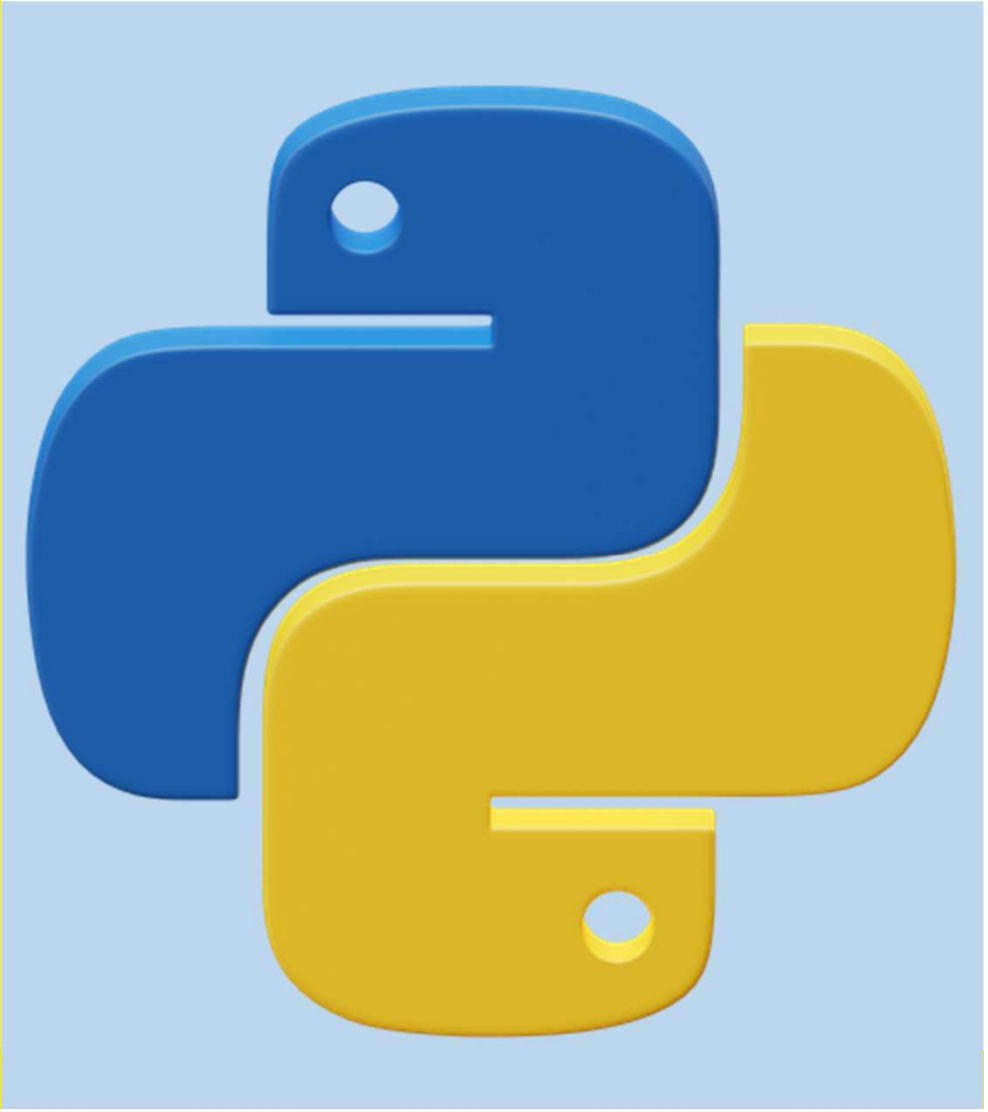
University of Applied Sciences and Arts Northwestern Switzerland
School of Business

Advanced Programming

NiceGUI Framework (1)



Rainer Telesko

Table of content

What Problem NiceGUI Solves

- Building web UIs usually requires multiple languages.
- Frontend frameworks add complexity for backend developers.
- NiceGUI allows full-stack development in Python.
- Python framework for building web-based graphical interfaces.
- Runs a web server and controls UI from Python.
- Browser is a rendering client, not the logic owner.

What NiceGUI Is Not

- Not a JavaScript replacement for custom UIs
- Not a low-level frontend framework
- Not ideal for large consumer-facing apps

Key Design Philosophy

- Server-driven UI
- Python-first development
- Simplicity over frontend flexibility

Mental Model for Beginners

- Think of the browser as a remote screen
- The server decides what the UI looks like
- User actions are sent back to Python

NiceGUI in more detail (1)

- What it is: NiceGUI is an open-source Python framework for building web-based user interfaces with minimal frontend work (https://nicegui.io/)
- Core idea: You write pure Python; NiceGUI handles the HTML/CSS/JS under the hood.
- Backend-first: It's built on FastAPI (modern Python web framework used to build APIs and web backends quickly and efficiently) and WebSockets, enabling real-time, reactive UIs without manual JavaScript.
- UI components: Provides ready-made components (buttons, inputs, tables, charts, dialogs, layouts) that update automatically when Python state changes.
- Frontend tech: Uses Vue.js and Quasar internally, but you don't need to know them to be productive.
- Use cases: Dashboards, internal tools, data apps, prototypes, admin panels, and ML/AI demos.

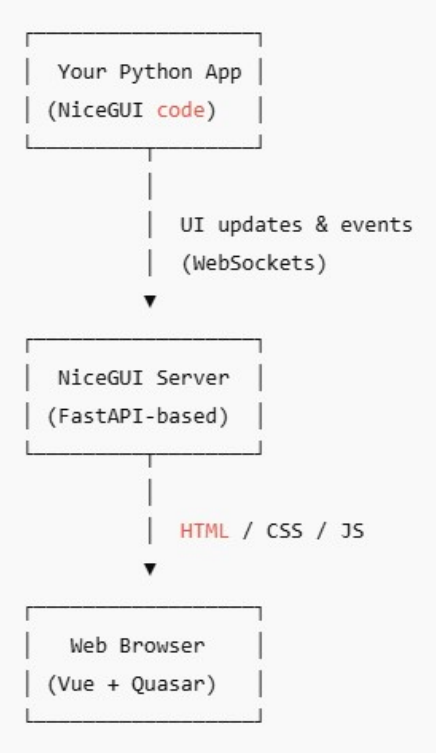
NiceGUI in more detail (2)

- Deployment: Runs as a normal Python app; can be containerized with Docker and deployed anywhere FastAPI apps run.
- Extensibility: You can add custom HTML, CSS, and JavaScript when needed.
- Learning curve: Very beginner-friendly for Python developers, especially compared to traditional frontend frameworks.
- License: Open source (MIT).
- Philosophy: Python defines the UI → the browser shows it → user actions go back to Python → Python updates the UI live

NiceGUI – comparison with other frameworks

Feature / Framework	NiceGUI	Streamlit	Dash	Gradio
Primary focus	General-purpose web apps & dashboards	Data apps & quick demos	Analytical dashboards	ML/AI model demos
Programming model	Event-driven, reactive UI	Script reruns top-to-bottom	Callback-based	Function-in / UI-out
Frontend code needed	❌ None (optional JS/CSS)	❌ None	❌ None (but more config)	❌ None
Underlying backend	FastAPI + WebSockets	Custom Streamlit server	Flask / FastAPI	FastAPI
UI flexibility	⭐⭐⭐⭐ Very flexible	⭐⭐ Limited layouts	⭐⭐⭐ Structured	⭐⭐ Simple
Real-time updates	✅ Native (WebSockets)	⚠️ Limited	⚠️ Via callbacks	⚠️ Limited
State handling	Explicit & predictable	Can be tricky	Explicit callbacks	Simple, limited
Customization	High (HTML, JS, CSS)	Low–Medium	Medium	Low
Learning curve	Low for Python devs	Very low	Medium	Very low
Best for	Internal tools, dashboards, full apps	Exploratory data apps	Production analytics dashboards	Model demos & sharing
Production readiness	High	Medium	High	Medium
Deployment	Any FastAPI-compatible setup	Streamlit Cloud / custom	Enterprise-friendly	Hugging Face / custom

NiceGUI – the big picture



Philosophy:

Python defines the UI →
Browser shows it →
User actions go back to Python →
Python updates the UI live

Why Server-Driven UI Matters

- What is server-driven UI?
- Server: logic, state, UI definition
- Client: rendering and user input
- No business logic in the browser

- Advantages
- Single source of truth
- Easier debugging
- Less frontend state complexity

NiceGUI – the workflow

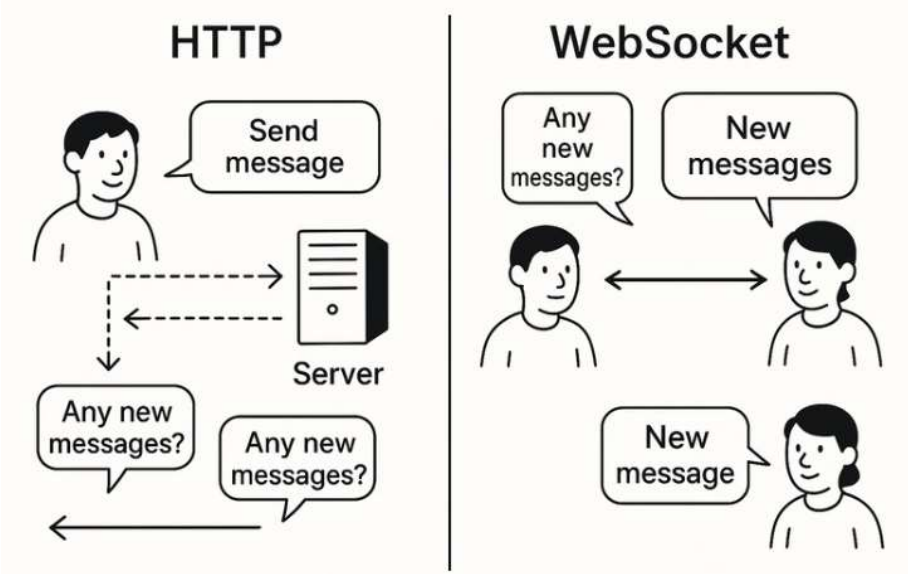
1. You write Python UI code
2. NiceGUI starts a web server
3. The browser renders the interface
4. User actions are sent back to Python
5. Python updates the UI in real time

What is a WebSocket?

- A WebSocket is a way for a Python server and a browser (or another client) to keep a single, long-lived connection open so they can send messages back and forth instantly.
- Why it exists
- Normal HTTP works like this: Client asks → server replies → connection ends.
- If the server has new info later, it can't "push" it unless the client asks again.
- WebSockets solve that by keeping a live channel open
- Client and server can both send messages any time (two-way, real-time).
- Mental model
- HTTP = texting someone, waiting for a reply each time
- WebSocket = staying on a phone call where either person can speak whenever

HTTP vs. WebSocket

Feature	HTTP (JSON)	WebSockets
Connection Model	Request-response	Persistent, open connection
Data Format	JSON	JSON / binary / text
Server-Initiated Msgs	No	Yes
Overhead	High (headers per request)	Low (single handshake)
Best For	CRUD, APIs	Real-time updates

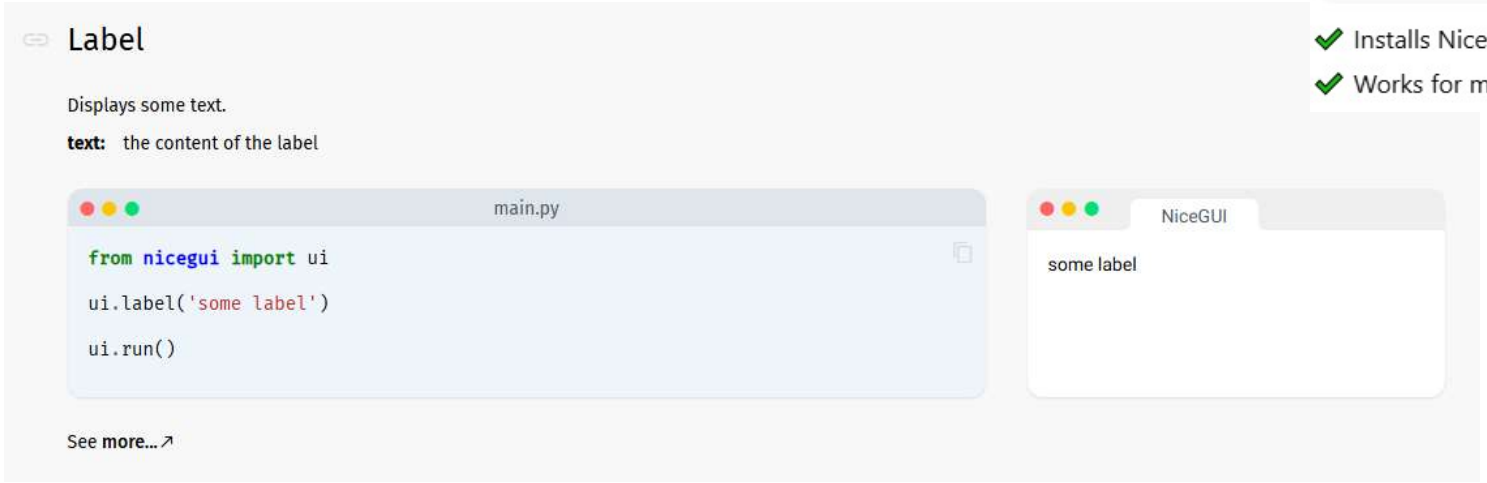


Use WebSockets when …

- You need real-time updates
- The server must push data to the client
- You're building chat apps, live games, or notifications

NiceGUI documentation

- URL: https://nicegui.io/documentation
- Explains all UI elements in detail and shows the look&feel
- A must for bookmarking!



Installation

```
pip install nicegui
```

✔ Installs NiceGUI and all required dependencies
✔ Works for most users

Minimum example – «Hello World»

```
from nicegui import ui

ui.label('Hello NiceGUI!')
ui.run()
```

`ui.run()` is what *starts* the NiceGUI application.
It doesn't create the UI itself - it boots the runtime that makes everything work.

NiceGUI Elements – What They Are

- NiceGUI elements are Python objects that represent UI components
- They define what the user sees and interacts with
- Each element has a clear purpose and responsibility

Text & Display Elements

- `ui.label` – display static or dynamic text
- `ui.markdown` – render formatted text
- `ui.html` – embed raw HTML when needed
- `ui.image` – display images

Input Elements

- `ui.input` – single-line text input
- `ui.textarea` – multi-line text input
- `ui.number` – numeric input
- `ui.checkbox` – boolean input
- `ui.switch` – toggle input

Selection Elements

- `ui.select` – choose from predefined options
- `ui.radio` – exclusive option selection
- `ui.slider` – numeric range selection
- `ui.toggle` – compact option switching

Action Elements

- `ui.button` – trigger actions and events
- `ui.link` – navigate to URLs
- `ui.menu` – group related actions
- `ui.context_menu` – actions bound to elements

Layout & Structure Elements

- `ui.row` – horizontal layout container
- `ui.column` – vertical layout container
- `ui.card` – grouped content with visual boundary
- `ui.expansion` – collapsible content section
- `ui.separator` – visual separation

Feedback & Status Elements

- `ui.notify` – short user notifications
- `ui.spinner` – indicate background activity
- `ui.progress` – show task progress
- `ui.badge` – highlight status or counters

Data Display Elements

- `ui.table` – structured tabular data
- `ui.tree` – hierarchical data display
- `ui.json_editor` – inspect and edit JSON data
- `ui.log` – display logs or streams

Page & Navigation Elements

- `ui.page` – define application pages
- `ui.tabs` – switch between views
- `ui.stepper` – guided multi-step workflows
- `ui.breadcrumbs` – navigation context

Element Properties

- Properties define appearance and behavior
- Example: text, value, enabled, …
- Properties can change at runtime

```python
from nicegui import ui

label = ui.label('Watch my color change!').style('color: black')

def change_color():
    label.style('color: red')  # update color property

ui.button('Make it red', on_click=change_color)

ui.run()
```

```python
from nicegui import ui

button = ui.button('Disabled button')
button.enabled = False  # initial state

def toggle():
    button.enabled = not button.enabled  # toggle property

ui.button('Toggle enable', on_click=toggle)

ui.run()
```

Events Explained

- Events represent user interactions
- Examples: button click, input change, …
- Handled by Python callbacks
- NiceGUI is event-driven, so callbacks are the core programming model.
- Callback
  - In NiceGUI, a callback is a Python function that is automatically executed in response to a user interaction or event in the UI.
  - A callback is “what should run when something happens.”
  - Executed on the server
  - Can update UI and state

Events Explained

Named function

```python
def handle():
    print('clicked')

ui.button('Click', on_click=handle)
```

Lambda (inline)

```python
ui.button('Click', on_click=lambda: print('clicked'))
```

Lambda with arguments

```python
def greet(name):
    print(name)

ui.button('Hi', on_click=lambda: greet('Alice'))
```

Event types

- `on_click` → triggered when a user clicks a button or clickable element
- `on_change` → triggered when the value of inputs or sliders changes
- `on_value_change` → similar to on_change, used by some widgets for value updates
- `on_keydown` → triggered when a key is pressed (keyboard input)
- `on_mouseover` / `on_mouseout` → triggered when the mouse enters or leaves an element
- `on_upload` → triggered when a file is uploaded

Understanding State

- State = the data your app remembers
  - A list of tasks, a counter value, a checkbox being checked
  - 👉 In NiceGUI, state usually lives on the server (Python)

- Why State Matters
  - UI is just a reflection of state
  - Events modify state, not the UI directly
  - You must manually update or re-render the UI
  - Clear flow makes apps easier to understand:
    User action → change state → update UI

```python
tasks = []  # this is state

def add_task():
    tasks.append(input.value)  # state changes
    refresh_ui()               # UI updates
```

Options for updating the UI - automatic updates

– Direct binding
  - updating a UI element directly

```
label.text = "New value"
```

– Binding
  - automatically connect data and UI
  - bindings in NiceGUI are uni- or bidirectional

NiceGUI bindings follow a consistent pattern:

- `bind_X` → 🔁 *two-way*
- `bind_X_from` → ➡️ *data → UI*
- `bind_X_to` → ➡️ *UI → data*

Binding options in NiceGUI

Method	Direction	What it binds	UI → Data?	Data → UI?	Typical Use
`bind_value`	🔁 Two-way	`.value` property	✅	✅	Inputs, sliders, switches
`bind_value_from`	➡️ One-way	`.value`	❌	✅	Display derived values
`bind_value_to`	➡️ One-way	`.value`	✅	❌	Push UI changes elsewhere
`bind_text`	🔁 Two-way	`.text`	✅	✅	Editable text elements
`bind_text_from`	➡️ One-way	`.text`	❌	✅	Labels, static text
`bind_text_to`	➡️ One-way	`.text`	✅	❌	Rare, manual propagation
`bind_visibility`	🔁 Two-way	visibility	✅	✅	Show/hide toggles
`bind_visibility_from`	➡️ One-way	visibility	❌	✅	Conditional display

Options for updating the UI - manual updates

- Self-created refresh (imperative):
  - Writing a function that updates/rebuilds the UI (often using `.clear()`)
  - You decide: what to delete, what to rebuild and when to call it

```
def refresh_tasks():
    container.clear()
    for t in tasks:
        ui.label(t)
```

- `@ui.refreshable` (declarative)
  - You define a render function and trigger it
  - NiceGUI reruns the function and handles updating the UI

```
@ui.refreshable
def view():
    for t in tasks:
        ui.label(t)

view.refresh()
```

“Silencing the editor” in VS Code

Changes in `settings.json`

```
"editor.wordBasedSuggestions": "off",
"editor.inlineSuggest.enabled": false,
"editor.snippetSuggestions": "none",

"github.copilot.enable": {
  "*": false,
  "python": true,
  "plaintext": false,
  "markdown": false,
  "scminput": false
},
"github.copilot.nextEditSuggestions.enabled": false
```

This block now cleanly groups all noise-reduction settings:

`"editor.wordBasedSuggestions": "off"`
→ kills random word spam

`"editor.inlineSuggest.enabled": false`
→ removes ghost text

`"editor.snippetSuggestions": "none"`
→ removes snippet clutter

Copilot settings →
stop AI suggestions except where you want them

Summary

- NiceGUI: a Python-based web framework for building interactive web applications with minimal frontend code.
- It allows developers to create user interfaces using pure Python, without needing to write HTML, CSS, or JavaScript directly.
- Built on FastAPI and Vue.js: uses FastAPI for the backend and Vue.js for reactive frontend components.
- Declarative UI in Python: create buttons, inputs, charts, tables, and layouts using simple Python syntax.
- Real-time interaction: automatic two-way communication between frontend and backend via WebSockets.
- Easy deployment: can run locally, on servers, or inside Docker containers.

Exercises

- Select screens for implementation (should be related to user stories)
- Define focused screens and navigation with a wireframing tool (e.g. Figma)
- See for tools https://cpoclub.com/tools/best-mockup-tools/